

MCA Semester 1	Subject : Advanced Data Structures Lab
Name : Mukund Gangurde	Topic: Applications of Linked Lists 1) List Based Queues 2) List Based Priority Queues
Roll No. : MCA2511	Date : 21-11-2025

1) List Based Queues.

Code:

```
import java.util.Scanner;
```

```
class QNode
```

```
{
    int data;
    QNode next;

    public QNode(int d)
    {
        data = d;
        next = null;
    }
} //end of QNode
```

```
//List based Queue Template
```

```
class LQueue
```

```
{
    QNode front;
    QNode rear;

    public LQueue()
    {
        front = null;
        rear = null;
    } //Default constructor

    //Enqueue
    public void Enqueue(int x)
    {
        //Make a new Node
        QNode t = new QNode(x);

        //First node in the Queue
        if(front == null)
        {
            front = t;
            rear = t;
        }
    }
}
```

```
        else    //Any other Node
        {
            rear.next = t;
            rear = t;
        }
    } //end of Enqueue

    //Dequeue
    public void Dequeue()
    {
        QNode tmp = front;

        //Check Empty Queue
        if(tmp == null)
        {
            System.out.println("Queue Underflowed...");
            return;
        }

        if(front == rear && front.next == null)
        {
            front = null;
            rear = null;
        }
        else
        {
            front = front.next;
        }
        System.out.println("Element Dequeued : " + tmp.data);
    } //end of Dequeue

    //PeekFront
    public void PeekFront()
    {
        //Check Empty Queue
        if(front == null)
        {
            System.out.println("Queue Underflowed...");
        }
        else
        {
            System.out.println("Element at Peek Front: " + front.data);
        }
    } //end of PeekFront

    //PeekRear
    public void PeekRear()
    {
        //Check Empty Queue
        if(front == null)
        {
            System.out.println("Queue Underflowed...");
        }
    }
}
```

```
        }
        else
        {
            System.out.println("Element at Peek Rear: " + rear.data);
        }
    }
} //end of PeekRear

//Display
public void Display()
{
    QNode tmp = front;

    //Check Empty Queue
    if(front == null)
    {
        System.out.println("Queue Underflowed...");
        return;
    }

    System.out.print("Queue Elements - ");
    while (tmp != null)
    {
        System.out.print(tmp.data + " ");
        tmp = tmp.next;
    }
    System.out.println();
} //end of Display
} //end of LQueue

//Menu Main class & method
class LQ
{
    public static void main(String[] args)
    {
        LQueue q = new LQueue();
        Scanner sc = new Scanner(System.in);
        int ch, x;

        do
        {
            System.out.println("\n\n1. Enqueue an element in Queue");
            System.out.println("2. Dequeue an element in Queue");
            System.out.println("3. Peek Front element in Queue");
            System.out.println("4. Peek Rear an element in Queue");
            System.out.println("5. Display element's in Queue");
            System.out.println("6. Exit\n");

            System.out.print("Enter your choice: ");
            ch = sc.nextInt();

            switch(ch)
            {
```

```
case 1:
    System.out.print("Enter your value element: ");
    x = sc.nextInt();
    q.Enqueue(x);
    q.Display();
break;

case 2:
    q.Dequeue();
    q.Display();
break;

case 3:
    q.PeekFront();
break;

case 4:
    q.PeekRear();
break;

case 5:
    q.Display();
break;

case 6:
    System.out.println("Exiting (...)");
break;

default:
    System.out.println("Incorrect choice :(|");
break;

    }
    } while (ch != 6);
} //end of psvm
} //end of LQ class
```

Output:

```
A:\MCA2511\DS_LAB>javac 17LQ.java
```

```
A:\MCA2511\DS_LAB>java LQ
```

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

```
Enter your choice: 1
```

```
Enter your value element: 12
```

```
Queue Elements - 12
```

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

```
Enter your choice: 1
```

```
Enter your value element: 13
```

```
Queue Elements - 12 13
```

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 1
Enter your value element: 5
Queue Elements - 12 13 5

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 1
Enter your value element: 6
Queue Elements - 12 13 5 6

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 1
Enter your value element: 45
Queue Elements - 12 13 5 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 5
Queue Elements - 12 13 5 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 3

Element at Peek Front: 12

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 4

Element at Peek Rear: 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 2
Element Dequeued : 12
Queue Elements - 13 5 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 2
Element Dequeued : 13
Queue Elements - 5 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 5
Queue Elements - 5 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 2
Element Dequeued : 5
Queue Elements - 6 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 2
Element Dequeued : 6
Queue Elements - 45

1. Enqueue an element in Queue
2. Dequeue an element in Queue
3. Peek Front element in Queue
4. Peek Rear an element in Queue
5. Display element's in Queue
6. Exit

Enter your choice: 2
Element Dequeued : 45
Queue Underflowed...

2) List Based Priority Queues

Code:

```
import java.util.Scanner;
```

```
class PQNode
```

```
{
```

```
    int data;
```

```
    int priority;
```

```
    PQNode next;
```

```
    public PQNode(int d, int p)
```

```
    {
```

```
        data = d;
```

```
        priority = p;
```

```
        next = null;
```

```
    }//Default Constructor
```

```
}//end of PQNode
```

```
class PQueue
```

```
{
```

```
    PQNode front;
```

```
    PQNode rear;
```

```
    public PQueue()
```

```
    {
```

```
        front = null;
```

```
        rear = null;
```

```
    }//Default Constructor
```

```
    //Enqueue in order of Priority
```

```
    public void Enqueue(int d, int p)
```

```
    {
```

```
        //Make a new Node
```

```
        PQNode t = new PQNode(d,p);
```

```
        //First Node in th PQ
```

```
        if(front == null)
```

```
        {
```

```
            front = t;
```

```
            rear = t;
```

```
            return;
```

```
        }
```

```
        //Traverse based on Priority
```

```
        PQNode tmp = front;
```

```
        PQNode prev = null;
```

```
        while(tmp != null && tmp.priority < t.priority)
```

```
        {
```

```
            prev = tmp;
```

```
        tmp = tmp.next;
    }

    //Insert node based on priority
    if(tmp == front)                //Front NNode Insertion
    {
        t.next = front;
        front = t;
    }
    else if(tmp == null)
    {
        rear.next = t;
        rear = t;
    }
    else
    {
        prev.next = t;
        t.next = tmp;
    }
} //end of Enqueue

//Dequeue
//Peek Front
//Peek Rear

//Display
public void Display()
{
    PQNode tmp = front;

    //Check Empty Queue
    if(front == null)
    {
        System.out.println("Queue Underflowed...");
        return;
    }

    System.out.println("\nPriority Queue contains---");
    System.out.println("Elements\tPriority- ");
    while (tmp != null)
    {
        System.out.println(tmp.data + "\t\t" + tmp.priority);
        tmp = tmp.next;
    }
    System.out.println();
} //end of Display
} //end of PQueue

class PQ
{
    public static void main(String[] args)
    {
```

```
PQueue p = new PQueue();
Scanner sc = new Scanner(System.in);
int ch,x,y;

do
{
    System.out.println("\n-----");
    System.out.println("-----Priority Queue-----");
    System.out.println("-----");
    System.out.println("1. Enqueue in Priority Queue");
    System.out.println("2. Dequeue from Priority Queue");
    System.out.println("3. Peek Front in Priority Queue");
    System.out.println("4. Peek Rear in Priority Queue");
    System.out.println("5. Display Priority Queue");
    System.out.println("6. Exit.....\n");

    System.out.print("Enter your choice: ");
    ch = sc.nextInt();

    switch(ch)
    {
        case 1:
            System.out.print("Enter your value: ");
            x = sc.nextInt();
            System.out.print("Enter your Priority: ");
            y = sc.nextInt();
            p.Enqueue(x,y);
            break;

        case 2:
            break;

        case 3:
            break;

        case 4:
            break;

        case 5:
            p.Display();
            break;

        case 6:
            System.out.println("Exiting ... :)");
            break;

        default:
            System.out.println("Incorrect choice :(");
            break;
    }
} while (ch != 6);
```

```
}//end of psvm  
}//end of PQ
```

Output:

```
A:\MCA2511\DS_LAB>javac 171PQ.java
```

```
A:\MCA2511\DS_LAB>java PQ
```

```
-----  
-----Priority Queue-----  
-----
```

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

```
Enter your choice: 1  
Enter your value: 12  
Enter your Priority: 4
```

```
-----  
-----Priority Queue-----  
-----
```

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

```
Enter your choice: 1  
Enter your value: 56  
Enter your Priority: 1
```

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 1
Enter your value: 56
Enter your Priority: 6

-----Priority Queue-----

1. Enqueue in Priority Queue
2. Dequeue from Priority Queue
3. Peek Front in Priority Queue
4. Peek Rear in Priority Queue
5. Display Priority Queue
6. Exit.....

Enter your choice: 5

Priority Queue contains---

Elements	Priority-
56	1
12	4
56	6